

BCC2001

— ORIGIN
'warn'.

monitor •

thing. 2 a software
duties. 3 a software
picture for use in
process which

Diego Bertolini

diegobertolini@gmail.com

Aula

- Estrutura de Dados Composta
 - Strings em Linguagem C

String: um tipo “especial” de vetor

Não existe o tipo “string” em C

No entanto, é óbvio que precisamos representar textos em C

Uma sequência de letras é similar a uma “string”

a	b	a	c	a	x	i	
0	1	2	3	4	5	6	7

Um tipo “especial” de vetor

Porém, como sabemos se chegamos ao final de uma “string”?

Convenção:

Uma string sempre termina com '\0'

Não é necessário que uma string utilize todos os espaços disponíveis no vetor!

Também não interessam os valores após o '\0'

a	b	a	c	a	x	i	\0	t	a	3	2	X
0	1	2	3	4	5	6	7	8	9	10	11	12

Um tipo “especial” de vetor

Implicações

Ao invés de utilizar apenas o tamanho do vetor com critério de parada, é necessário considerar a ocorrência do primeiro '\0'

- O que acontecer primeiro!

Exemplo: comparação de strings

a	b	a	c	a	x	i	\0
0	1	2	3	4	5	6	7

a	b	a	c	a	x	i	\0	t	a	3	2	X
0	1	2	3	4	5	6	7	8	9	10	11	12

Inicializando uma string

```
char str [12] = {'H','e','l','l','o', ' ', 'W','o','r','l','d', '\0'} ;
```

```
char str [] = {'H','e','l','l','o', ' ', 'W','o','r','l','d', '\0'} ;
```

```
char str [12] = {"Hello World"} ;
```

Lendo uma string

```
char str[20];  
scanf("%s");
```

Quando usamos a função `scanf()` para ler uma string, o símbolo `&` antes do nome **não é utilizado**. Colchetes também não são utilizados, pois queremos ler a string toda.

A função `scanf()` lê apenas strings digitadas sem espaços, ou seja, palavras.

scanf() e strings

Símbolo que corresponde a uma string:

`%s`

Para ler uma string, não preciso usar o &

O nome do vetor é uma referência à memória!

Exemplo:

```
char palavra[20];
```

```
scanf("%s", palavra);
```


scanf() e strings

No entanto, ao indicar uma posição de um vetor, passamos a indicar um valor ao invés de uma referência.

Se quisermos, podemos passar o endereço daquele valor:

Exemplo:

```
char palavra[20];
```

```
scanf("%s", &palavra[4]);
```

```
// String vai ser armazenada da posição 4 em  
diante
```

scanf() e strings

No entanto, usar apenas “%s” apresenta limitações!

Lê até encontrar um espaço em branco na string!

Solução:

```
scanf("%[^\\n]", palavra);
```

scanf() e strings

Solução:

```
scanf("%[^\\n]", palavra);
```

Lê qualquer letra que não seja uma nova linha!

'^' é símbolo para negação

'[' e ']' definem uma faixa de valores aceitos pelo '%s'

- `[^\\n]` == Qualquer letra que não seja um quebra de linha

Função gets()

Uma alternativa mais eficiente para leitura de uma string é a função `gets()`, a qual faz a leitura do teclado considerando todos os caracteres digitados (incluindo os espaços em branco) até encontrar uma tecla enter.

```
char str[20] ;  
gets(str) ;
```

Função fgets()

```
char *fgets(char * str, int tamanho, FILE *fp)
```

A função fgets() recebe três parâmetros:

- str: a string a ser lida ;
- Tamanho : o limite máximo de caracteres a serem lidos ;
- FP: a variável que está associada ao arquivo de onde a string será lida.

Função fgets()

Para ler do teclado, basta substituir FILE *fp por stdin, o qual representa o dispositivo de entrada-padrão (geralmente o teclado).

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char nome[30] ;
6      printf("Digite um nome: ");
7      fgets(nome, 30, stdin);
8
9
10     printf("O nome digitado: %s", nome);
11
12     return 0 ;
13
14
15 }
```

```
Digite um nome: Diego Bertolini Goncalves - DACOM - CM - 2014
O nome digitado: Diego Bertolini Goncalves - D
```

Função fgets()

Na Função fgets(), o caractere de nova linha ('\n') fará parte da string, o que não acontecia com o gets().

A função fgets() especifica o tamanho máximo da string de entrada

Limpando o buffer do teclado

```
setbuf(stdin, NULL)
```

Limpando o buffer do teclado

Podemos limpar o buffer do teclado (entrada-padrão) usando a função `setbuf(stdin, NULL)` antes de realizar a leitura de caracteres ou strings.

Basicamente, a função `setbuf()` preenche um buffer (primeiro parâmetro) com determinado valor (segundo parâmetro). No exemplo o buffer de entrada (`stdin`), ou seja, o teclado é preenchido com o valor vazio (`NULL`). Na linguagem C, a palavra `NULL` é uma constante-padrão que significa um valor nulo. Um buffer preenchido com `NULL` é considerado limpo/vazio.

Escrevendo na tela

printf()

```
char str[20] = {"TESTE"};  
printf("%s", str);
```

fputs()

Recebe dois parâmetros:

str: a string (array de caracteres) a ser escrita na tela ;

fp: a variável que está associada ao arquivo onde a string será escrita.

Escrevendo na tela

fputs(str, FILE *fp)

```
1  #include <stdio.h>
2  #include <string.h>
3
4  int main() {
5      char nome[30] ;
6      printf("Digite um nome: ");
7      fgets(nome, 30, stdin);
8
9      fputs(nome, stdout);
10
11     return 0 ;
12
13
14 }
15
```

Funções para string

strlen()

strcmp() e strncmp()

strcpy() e strncpy()

strcat() e strncat()

atoi()

atof()

strstr()

strlen()

Retorna o tamanho de uma string

Por exemplo:

```
char palavra[30] = "abcd";  
int tamanho;  
tamanho = strlen(palavra);  
printf("O tamanho da palavra %s é %d", palavra, tamanho);
```

strlen()

```
int strlen(char texto[]) {  
    int i;  
    for (i = 0; texto[i] != '\0'; i++);  
    return i;  
}
```

strnlen()

```
int strnlen(char texto[], int tamanho_maximo) {  
    int i;  
    for (i = 0; i < tamanho_maximo && texto[i] !=  
        '\0'; i++);  
    return i;  
}
```


strcmp()

Compara duas strings

Por exemplo:

```
char palavra1[30] = "abcd";  
char palavra2[30] = "wxyz";  
int resultado;  
resultado = strcmp(palavra1, palavra2);  
if (resultado == 0) {  
    printf("As palavras são iguais");  
} else if (resultado > 0) {  
    printf("A palavra1 é maior do que a palavra2");  
} else if (resultado < 0) {  
    printf("A palavra1 é menor do que a palavra2");  
}
```

strcmp()

```
int strcmp(char texto1[], char texto2[]) {
    int i;
    for (i = 0; texto1[i] == texto2[i] &&
        texto1[i] != '\0' && texto2[i] != '\0'; i++);

    if (texto1[i] == '\0' && texto2[i] == '\0') {
        return 0;
    } else {
        return texto1[i] - texto2[i];
    }
}
```

strncmp()

Compara duas strings (considerando apenas “n” letras)

Por exemplo:

```
char palavra1[30] = "abcd";
char palavra2[30] = "abcde";
int resultado;
resultado = strncmp(palavra1, palavra2, 3);
if (resultado == 0) {
    printf("As palavras definidas pelas 3 primeiras
        letras de cada palavra são iguais");
} else if (resultado > 0) {
    printf("A palavra definidas pelas 3 primeiras
        letras da palavra1 é maior do que aquela da
        palavra2");
} else if (resultado < 0) {
    printf("A palavra definidas pelas 3 primeiras
        letras da palavra1 é menor do que aquela da
        palavra2");
}
```

strncmp()

```
int strncmp(char texto1[], char texto2[], int
tamanho_maximo) {
    int i;
    for (i = 0; i < tamanho_maximo
        && texto1[i] == texto2[i]
        && texto1[i] != '\0'
        && texto2[i] != '\0'; i++);

    if (i == tamanho_maximo
        || (texto1[i] == '\0'
            && texto2[i] == '\0')) {
        return 0;
    } else {
        return texto1[i] - texto2[i];
    }
}
```

strcpy()

Copia o conteúdo de uma string para outra string

É como se fosse um comando de atribuição, mas e strings

Por exemplo:

```
char palavra1[30] = "abcd";  
char palavra2[30];  
strcpy(palavra2, palavra1);  
printf("Palavra2: %s", palavra2);
```

strncpy()

Copia as n primeiras letras de uma string para outra string

Por exemplo:

```
char palavra1[30] = "abcd";
```

```
char palavra2[30];
```

```
strncpy(palavra2, palavra1, 2);
```

```
printf("Palavra2: %s", palavra2); // Resultado: ab
```

strcat()

Adiciona (concatena) o conteúdo de uma string para outra string

O que acontece com o '\0' da primeira string?

Por que?

Por exemplo:

```
char palavra1[30] = "efgh";
```

```
char palavra2[30] = "abcd";
```

```
strcat(palavra2, palavra1);
```

```
printf("Palavra2: %s", palavra2); // Resultado:  
abcdefgh
```

strncat()

Adiciona (concatena) as primeiras n letras de uma string para outra string

Por exemplo:

```
char palavra1[30] = "efgh";  
char palavra2[30] = "abcd";  
strncat(palavra2, palavra1, 2);  
printf("Palavra2: %s", palavra2); // Resultado: abcdef
```


atoi()

Converte uma string para inteiro

Por exemplo:

```
char palavra[30] = "123";  
int numero;  
numero = atoi(palavra);  
printf("Número: %d", numero); // Resultado: 123
```

Caso não consiga converter (por exemplo, não é um número), o resultado é zero.

No entanto, talvez a palavra convertida seja o número zero!

atof()

Converte uma string para ponto flutuante

Por exemplo:

```
char palavra[30] = "123.456";  
double numero;  
numero = atof(palavra);  
printf("Número: %g", numero); // Resultado: 123.456
```

Caso não consiga converter (por exemplo, não é um número), o resultado é zero.

No entanto, talvez a palavra convertida seja o número zero!

Exercícios

- 1) Faça um programa que seja possível inserir uma palavra de até 30 caracteres e demonstre:
 - a) Palavra Invertida:
 - b) Quantidade de letras 'aeiou';

Dúvidas, Críticas ou Sugestões

diegobertolini@gmail.com